DSSE in JaxAHT: A Drone Swarm Search-and-Rescue AHT Benchmark

Lain (Zelal) Mustafaoglu

The University of Texas at Austin
zsm@utexas.edu

April 2026

Abstract

The **Drone Swarm Search Environment** (**DSSE**) [5] is a cooperative multi-drone search-and-rescue task with the kind of structure that should make a useful Ad Hoc Teamwork (AHT) benchmark: continuous observations, partial observability, stochastic target drift, a long episode horizon. This writeup describes a JAX reimplementation of DSSE for the JaxAHT framework and evaluates two ego methods on it (PPO ego and LIAM ego, the teammate-modeling baseline that already ships with JaxAHT) across four experiments, plus an integration check for three additional partner generators. For the benchmark to be meaningful, the rescue rule needs a small change: a target counts as rescued only when at least NDR drones search the same cell on the same step, where NDR is a single integer hyperparameter (number of drones required to rescue). NDR= 1 recovers the original rule; NDR= 2 is the configuration evaluated below, where solo strategies no longer work and teammate modeling has something to do. Against a 300-partner Fictitious Co-Play population at 10^7 environment steps, LIAM ego beats PPO ego by 4.3× in mean final return and 13× in best per-iteration return.

1 Introduction

Ad Hoc Teamwork (AHT) is the problem of cooperating with teammates you have not coordinated with in advance [1]. The standard recipe is to train an ego agent against a pool of partner policies, deploy it against (ideally) unseen partners drawn from the same distribution, and give the ego some way to model the partner from interaction history. A method that does this well should beat a baseline that ignores the partner entirely, but that comparison only makes sense if the task actually requires the partner: on a task where one agent can win alone, ignoring the partner is already a near-optimal strategy and a teammate model has nothing to learn from.

The **Drone Swarm Search Environment** (**DSSE**) [5] is a cooperative multi-drone search-and-rescue task on a grid. Drones move through a probability-of-detection field looking for moving targets and search a cell to attempt a rescue. Continuous-valued observations, partial observability, stochastic target drift, a long episode horizon, and a search-and-rescue payoff that maps cleanly onto realistic multi-drone deployments make **DSSE** a natural fit for AHT in almost every respect. The one place it does not fit out of the box is the rescue rule: in stock DSSE a single drone can locate and rescue a target on its own, so an ego that never models its teammate is already near-optimal.

This writeup describes a JAX reimplementation of **DSSE** for the JaxAHT framework [4], together with an experimental validation against two ego methods. PPO ego is the non-teammate-modeling baseline (a vanilla recurrent PPO ego with no teammate head); LIAM ego [2] is the teammate-modeling baseline that already ships with JaxAHT. For the benchmark to be meaningful, the rescue rule needs a small change: a target now counts as rescued only when at least NDR drones search the same cell on the same step, where NDR is a single integer hyperparameter (number of drones required to rescue). NDR= 1 recovers the original rule exactly; NDR= 2 rules out solo strategies because both drones now have to be on the same cell at the same time and both have to choose to search. Target drift, the probability-of-detection model, and the episode horizon are all untouched, so every existing DSSE configuration still runs.

The validation runs four experiments, three at NDR= 2 and one at NDR= 1 as a no-coordination control: an encoder-ablation negative control, a no-coordination sanity check at NDR= 1, a 3-partner held-out evaluation, and a 300-partner Fictitious Co-Play (FCP) population, followed by a cross-experiment summary. LIAM beats PPO in every measured experiment. The headline number is on the FCP run: 4.3× in mean final return and 13× in best per-iteration return, over 3 ego seeds at 10^7 environment steps each. The same direction shows up in every other experiment, and collapses to 1.21× when the rescue rule is reset to NDR= 1. A fifth experiment is an integration check (not a competitive com-

parison) confirming that three more partner generators (BRDiv, CoMeDi, LBRDiv) compile and run end to end on **DSSE**. The pure-JAX backend reaches about 27,000 environment steps per second at NUM_ENVS= 64 on one RTX 6000 Ada.

2 Background

In the AHT setting [1], an ego agent trains against a pool of partner policies and is evaluated against (ideally) unseen partners from the same distribution. Two ego methods anchor every result in this writeup. The non-teammate-modeling baseline is PPO ego as shipped in JaxAHT: a standard PPO loop on top of an S5 state-space actor, so the actor has a recurrent memory over the observation history but no dedicated partner-modeling machinery. The teammate-modeling method is LIAM [2], which JaxAHT ships with an MLP actor plus a small encoder head that takes the recent partner action history and produces a latent vector intended to represent "what kind of partner is this." The actor receives that latent vector as additional input. During training, the encoder is supervised against the ground-truth partner identity through a standard classification loss in addition to the PPO objective; at evaluation time the classification target is gone, but the encoder still produces its latent and the actor still conditions on it.

PPO ego and LIAM ego as shipped are not architecturally matched: PPO ego has an S5 recurrent backbone and no encoder, LIAM ego has an MLP backbone and an explicit partner encoder. The encoder-ablation experiment in Section 5.1 is still clean because it stays *within* the LIAM architecture (frozen random encoder versus trained encoder, both on MLP). And the no-coordination control in Section 5.2 rules out a pure architectural explanation: if LIAM beat PPO only because MLP-with-encoder were a stronger architecture than S5, the gap would not depend on the rescue rule, but it collapses from 4.3–12.9× at NDR= 2 to 1.21× at NDR= 1. A matched-backbone version of the comparison is listed in future work.

The largest-pool experiment in this writeup uses Fictitious Co-Play [3], which builds a partner population by training independent self-play agents and checkpointing each one at multiple training iterations. The resulting pool spans early-, mid-, and late-stage behaviors rather than only final policies, which gives the ego a wider behavioral range to learn against.

3 The DSSE task

Task. DSSE [5] is a cooperative search-and-rescue task on a $g \times g$ grid. n_d drones must locate n_t moving targets whose positions evolve under a stochastic drift model (a wind-direction vector plus a dispersion schedule). Each drone has nine discrete actions (eight single-cell moves plus SEARCH) and a partial observation consisting of its own cell index, the probability-of-detection map updated in place every step, and the positions of the other drones. In the original rule, a SEARCH on the target cell triggers a stochastic detection via a per-cell probability of detection (POD). Episodes terminate when all targets are rescued or the timestep limit is reached.

The rescue rule. The benchmark adds one integer hyperparameter, NDR, and modifies the rescue rule so that a target is rescued at step t only if at least NDR drones execute SEARCH on the target cell at step t and the POD draw succeeds. NDR= 1 recovers the original DSSE SAR rule exactly; NDR$\geq$ 2 makes a single drone insufficient to complete a SAR task. The drift model, the POD, the grid size, and the episode horizon are unchanged, so every existing DSSE configuration still runs. Figure 1 shows three rollouts on the same seed and target spawn: NDR= 1 solo (rescued), NDR= 2 solo (no rescue, ego chases the drifting target until the horizon), and NDR= 2 with both drones cooperating (rescued). The only thing that changes is whether the teammate cooperates.

Implementation. The DSSE environment is reimplemented as a pure-JAX module at `envs/dsse/dsse_jax.py`, with JIT-compiled `reset`, `step`, and `get_obs`, and registered in the JaxAHT environment registry so the existing ego and MARL trainers pick it up without modification. Partner pre-training runs through the same backend via `teammate_generation/run.py`; no PettingZoo subclass is needed, and the `DSSE` PyPI package is not a runtime dependency of any script in this PR.

Default configuration. The default configuration fixes $g = 7$, NDR= 2, and an episode horizon of 100 steps. The JaxAHT ego pipeline asserts `num_agents == 2`, and the teammate-generation methods (FCP, BRDiv, CoMeDi, LBRDiv) call into that pipeline as a subroutine, so every partner pool and every ego run reported in this writeup is at $n_d = 2$, $n_t = 1$. The only place $n_d > 2$ is currently supported is the raw `marl/run.py` self-play loop, used in Section 7 at $n_d = 4$, $n_t = 2$ for the throughput benchmark only.

Unit test. The unit test at `tests/test_dsse_wrapper.py` rolls out 32 envs for 50 steps each with drone 0 acting uniformly at random and drone 1 pinned to action 0 (LEFT). Drone 1 never executes SEARCH, so at NDR= 2 only drone 0 can count toward the two-searcher rule, and a single drone cannot satisfy it. The test sees 0/32 positive-reward envs on every run.

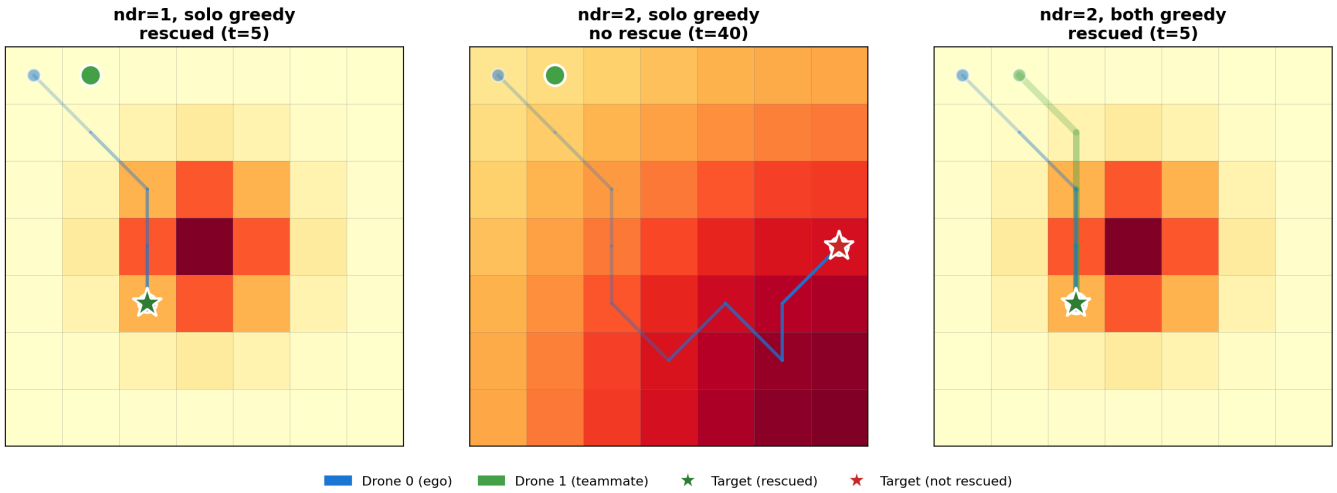


Figure 1: Three rollouts with the same seed and target spawn. Left: NDR= 1, the ego drone (blue) walks to the target alone and rescues it at t=5 while the teammate (green) idles. Centre: NDR= 2, same solo ego, same idling teammate; the ego reaches the target cell and SEARCHes repeatedly but the rescue never fires, then chases the drifting target until the horizon. Right: NDR= 2, both drones walk to the target together and rescue it at t=5. The greedy used here cheats with oracle target access for visualization; the trained policies in Section 5 have to infer the target from the probability map. This is generated by `scripts/visualize_dsse_coordination_demo.py`.

4 Experimental setup

4.1 Training settings

All ego training uses PPO-style on-policy updates with $\gamma = 0.99$, GAE $\lambda = 0.95$, clip range 0.2, and Adam at learning rate $3 \cdot 10^{-4}$ (the value in every `dsse.yaml` algorithm config). PPO ego is a vanilla PPO ego on an S5 recurrent actor with no teammate head. LIAM ego uses an MLP actor plus the LIAM encoder head, with the encoder supervised against the ground-truth partner identity through a classification loss in addition to the PPO objective. The two methods do not share an actor backbone: PPO ego is S5, LIAM ego is MLP. See Background for the full architectural description and the Limitations list for the matched-backbone future-work note. Ego training runs for 10^7 environment steps per seed at NUM_ENVS= 32. MARL pre-training (the FCP partner generation pass) runs for 10^7 steps at NUM_ENVS= 64. Final return is reported as the average of the last three checkpoints, which smooths end-of-training noise without cherry-picking an iteration.

The DSSE wrapper uses the `share_rewards=True` convention: it *sums* the per-drone reward array and hands the sum to every agent, so each agent receives the full team reward. The LBF wrapper instead *averages* the per-drone array, so each agent receives the mean. With n_{drones} drones, the DSSE reward scale is $n_{\text{drones}}\times$ the LBF scale, and every reported number in this document is on the DSSE scale.

4.2 Partner pools

The 300-partner FCP pool is generated by `teammate_generation/run.py` with `algorithm=fcp/dsse`: 3 FCP seeds $\times$ 20 agents per seed $\times$ 5 training-time checkpoints per agent, for 300 distinct partners shared across all ego seeds. The smaller 3-partner pools used in the encoder ablation, the population training run, and the held-out evaluation are drawn from the same distribution at 3 partners sampled from a single FCP seed.

4.3 Per-experiment script map

Each experiment is reproducible by running a single script from the repository root, listed in Table 1. The 3-partner population training script (`run_phase_c_population.sh`) does not have its own results subsection because the same orbax checkpoints are consumed by the 3-partner held-out evaluation in Section 5.3; its trained-partner returns populate the left column of Table 4. Auxiliary scripts for figures (`dsse_metrics.py`, `visualize_aht_comparison.py`, `visualize_dsse_episode.py`, `visualize_dsse_coordination_demo.py`, `run_dsse_scaling.sh`) are wrapped by `run_all_gpu.sh`; their outputs and Sacred logs live under `writeup/logs/` and `writeup/plots/`.

5 Results

Four LIAM-versus-PPO experiments are reported below, beginning with the encoder-ablation negative control and

Table 1: Reproduction scripts. Phases A, B, D, and E are the four LIAM-vs-PPO experiments; Phase C is the supporting partner training run consumed by Phase D; Phase F is the BRDiv / CoMeDi / LBRDiv integration check. All paths relative to `scripts/`; raw Sacred JSONs land under `writeup/logs/`.

Condition	Script
Encoder-ablation negative control	`run_phase_a_validity.sh`
No-coordination control (NDR= 1)	`run_phase_b_10seed.sh`
3-partner population training	`run_phase_c_population.sh`
3-partner held-out evaluation	`run_phase_d_heldout.sh`
300-partner Fictitious Co-Play	`run_phase_e_fcp.sh`
BRDiv / CoMeDi / LBRDiv integration check	`run_phase_f_brdiv_comedi_lbrdiv.sh`

ending with the 300-partner Fictitious Co-Play (FCP) run. A fifth experiment is the integration check for BRDiv, CoMeDi, and LBRDiv. Raw Sacred logs live under `writeup/logs`.

5.1 Encoder-ablation negative control

The first thing the benchmark has to rule out is that LIAM only beats PPO because it has more parameters or a slightly different architecture. The ablation here trains an ego with the full LIAM architecture intact but freezes the teammate encoder at random initialization and never runs a gradient through it. The frozen-encoder ego therefore has the same parameter count as trained LIAM, the same MLP actor backbone, the same actor input shape, and the same training pool; the only thing that changes is whether the encoder is allowed to learn. If the encoder were not doing real work, frozen LIAM and trained LIAM should land in roughly the same place, since both would reduce to noisy MLP policies with some extra weights attached. They do not. Frozen LIAM goes to exactly zero on every seed; trained LIAM reaches roughly nine times the PPO baseline. The encoder is what is carrying the result, not the architecture around it.

Table 2: Encoder ablation on a 3-partner pool at NDR= 2, 5 seeds. Source: `writeup/logs/phase_a_*.json`.

Condition	Mean ± std
PPO ego	0.0355 ± 0.0435
LIAM ego, frozen encoder	0.0000 ± 0.0000
LIAM ego, trained encoder	$\mathbf{0.3094 \pm 0.2623}$

The trained-encoder LIAM per-seed returns are [0.6425, 0.0000, 0.0000, 0.4310, 0.4735]. Two of the five seeds went to exactly zero, the same way one of the three seeds in the FCP experiment will (Section 5.4). The ±0.2623 standard deviation is the spread of those two modes, not a noise band, and the mean stays roughly 9× PPO even under that pessimistic averaging.

5.2 No-coordination control (ndr= 1)

This is the matching test for the encoder ablation. If the LIAM-over-PPO gap is really being driven by the coordination requirement, then turning the requirement off (NDR= 1, which recovers stock DSSE) should make the gap go away. Any LIAM-over-PPO advantage that survived at NDR= 1 would have to come from somewhere other than coordination. Ten ego seeds are used in these experiments.

Table 3: No-coordination control at NDR= 1, 10 seeds. Source: `writeup/logs/phase_b_*.json`.

Method	Mean ± std	Rel to PPO
PPO ego	1.1078 ± 0.1718	(baseline)
LIAM ego	$\mathbf{1.3446 \pm 0.0923}$	1.21×

The absolute returns are much larger than in the other experiments because solo rescues are trivially available at NDR= 1 and PPO is now competent on its own. The LIAM-over-PPO ratio collapses to 1.21×: the LIAM advantage only shows up when the rule makes coordination necessary.

5.3 3-partner held-out evaluation

This experiment trains LIAM and PPO against a small 3-partner population and then evaluates them on both the training partners and a disjoint held-out partner set drawn from the same generator. A teammate-modeling advantage that disappears the moment the ego sees a partner it has not trained against is not worth much. 10 seeds are used for ego agent training.

LIAM keeps its advantage against held-out partners with only a modest drop from the trained-partner number, and PPO stays very small under either evaluation. The held-out ratio (6.8×) is smaller than the trained-partner ratio (12.9×), which is the right direction: LIAM's encoder is trained with a classification loss against the specific partner identities it has seen, so the clean version of the task it learns is "recognize this exact partner" rather than "recognize any partner drawn from the same generator." The fact that the held-out ratio

Table 4: 3-partner population, trained and held-out partner evaluations, 10 seeds. Source: `writeup/logs/phase_d_*.json`.

Method	Trained-partner return	Held-out partner return
PPO ego	0.0018 ± 0.0053	0.0032 ± 0.0040
LIAM ego	$\mathbf{0.0232 \pm 0.0304}$	$\mathbf{0.0219 \pm 0.0294}$
Ratio	12.9×	6.8×

is still roughly 7× suggests the encoder is picking up something that transfers to nearby partners, just less efficiently.

5.4 300-partner Fictitious Co-Play

This is the experiment with the largest partner pool in this writeup: a Fictitious Co-Play population of 300 distinct partners (3 seeds × 20 agents × 5 checkpoints), with the ego trained against the full pool for 10^7 environment steps and the result averaged over 3 ego seeds.

LIAM ego beats PPO ego by 4.3× in mean final return and 13× in best per-iteration return. The second LIAM seed shows how not every LIAM run learns a useful policy within 10^7 steps on a 300-partner pool at this ego capacity, and that zero seed drags LIAM's mean down by roughly 9× from its best iteration without closing the gap with PPO. The fix is more budget or more capacity, not a change to the task. Three ego seeds is also fewer than the other experiments (5 or 10); scaling to more seeds is left for future work.

5.5 BRDiv / CoMeDi / LBRDiv integration check

This experiment verifies that three additional teammate generation pipelines (BRDiv, CoMeDi, LBRDiv) integrate end-to-end with the DSSE task and the held-out evaluation harness. It is an integration check, not a competitive comparison: the partner training budget is $5 \cdot 10^6$ steps, the ego budget is $2.5 \cdot 10^6$ steps, and a single seed is run per method. The defaults for these methods are $2 \cdot 10^7$ partner steps and 10^7 ego steps, and DSSE at NDR= 2 is a sparse-reward coordination task that punishes undertrained partner pools.

The LBRDiv overall of exactly 0 is informative, not a bug. Inspecting the raw held-out tensors confirms that every one of the 384 evaluation episodes (6 partners × 64 episodes) ran the full 100-step horizon with `targets_found = 0` and `all_found = 0`. The LBRDiv ego seed converged to a degenerate "do nothing" policy within the $2.5 \cdot 10^6$ ego step budget. BRDiv and CoMeDi each had one partner in the seed-123 IPPO pool (`ippo_seed123_pop3_0`) that produced a non-zero return (0.0286 and 0.0181 respectively); the other five held-out partners were also flat zero. With one seed and a sparse-reward coordination task this is the variance floor, not a method comparison.

What this integration check actually establishes for the PR: BRDiv, CoMeDi, and LBRDiv configs pass schema validation; the partner training loops compile; the ego training loops compile; the orbax checkpoints serialize and reload through `common.save_load_utils.load_train_run`; and the held-out eval harness consumes them via the same code path used by FCP. This writeup does not establish any ranking of these methods against FCP on DSSE. Large-scale experiments ($2 \cdot 10^7$ partner steps, 10^7 ego steps, 3 seeds) are planned for future work.

5.6 Cross-experiment summary

Table 7 pulls all four experiments into one view. The LIAM-over-PPO direction is the same in every row. The encoder ablation is the cleanest test of whether encoder learning matters at all (because frozen LIAM and trained LIAM differ only in whether the encoder is allowed to update); the no-coordination control gives the narrowest gap. The two 3-partner rows show what happens when the partner pool is small but the eval set changes: trained-partner eval is the widest gap in the table at 12.9× because LIAM is trained specifically to recognize the partners it has seen, while the held-out eval drops to 6.8× because the encoder was never asked to transfer to unseen partners during training. The 300-partner FCP row is narrower than either 3-partner row (4.3×) because the encoder now has to distinguish two orders of magnitude more partners at the same training budget.

6 Scripted-policy validity probe

The numerical experiments above all rely on learned ego policies. As a sanity check that the rescue rule itself is doing what it claims, three scripted policies probe the default 7×7 task (n_d=2, n_t=1, NDR= 2), with the ego fixed at drone 0 and the teammate varied. The Coordinated scenario has both drones run a greedy controller toward the target, so they co-locate and SEARCH on the same step. The Antagonist scenario has the ego greedy toward the target while the teammate runs greedy toward the diagonally opposite corner, so the rescue rule never fires. The Random teammate scenario has the ego greedy toward the target while the teammate samples actions uniformly, so the rescue rule only fires on the rare random walks that land the teammate on the target cell exactly

Table 5: LIAM ego versus PPO ego on the 300-partner FCP population. 3 ego seeds ($\{5, 6, 7\}$), 10^7 environment steps each. Source: `writeup/logs/*_fcp_v1.json`.

Method	Per-seed final return	Mean ± std	Best per-iter
PPO ego	[0.0009, 0.0025, 0.0013]	0.0015 ± 0.0007	0.0043
LIAM ego	[0.0046, 0.0000, 0.0152]	$\mathbf{0.0066} \pm \mathbf{0.0064}$	**0.0577**

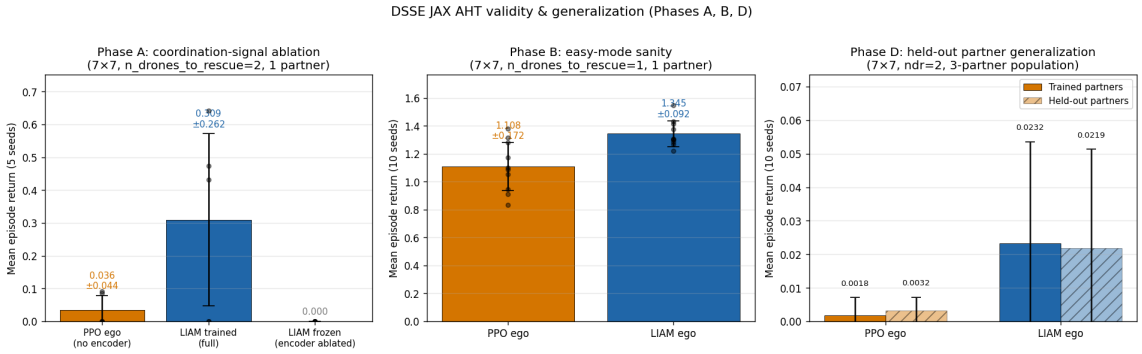


Figure 2: Overview across the four experiments. The LIAM-over-PPO direction is preserved in every panel, and the gap collapses cleanly only at NDR= 1 (the no-coordination control).

when the ego SEARCHes there. Over 200 seeded episodes per scenario, Coordinated rescues 200/200 (100%), Antagonist rescues 0/200 (0%), and Random teammate rescues 11/200 (5.5%). Figure 3 shows the final frame of each. The clean 100% to 0% split between Coordinated and Antagonist is the proof that the rescue rule depends on joint action, with no learned policy involved. This is generated by `scripts/visualize_aht_comparison.py`.

7 Throughput

A $5 \cdot 10^5$-step IPPO run on **DSSE** (7×7, 4 drones, 2 targets, NDR= 2, seed 42, one RTX 6000 Ada) finishes in 18.2 s at NUM_ENVS= 64, about 27,473 environment steps per second. A 10^7-step ego run finishes in about six minutes of wall clock at the same NUM_ENVS, which is what makes running the full sweep feasible at all. Throughput scales close to linearly with NUM_ENVS up to 64 and plateaus thereafter (Figure 4(b)). For reference, the same $5 \cdot 10^5$-step run on a PyTorch NAHT integration of the same task (a DSSE port I maintain on a fork of `carolinewang01/naht`) takes 364 s, which is 20× slower in end-to-end wall clock. This is not a matched architectural comparison: the PyTorch variant runs a recurrent GRU actor at `batch_size_run=1` where the dispatch overhead dominates, while the JAX variant runs an MLP actor in a vectorized loop. The iteration cost on JaxAHT is an order of magnitude lower than the same task on the `epymarl`-based NAHT integration at the configuration this writeup actually uses.

8 Limitations and future work

1. **Two-drone ego training and partner pools.** The JaxAHT ego pipeline asserts `num_agents == 2`, and the teammate-generation methods inherit this because they call into the ego trainer as a subroutine. Every partner pool and every ego run reported here is at 2 drones, 1 target. Lifting the assertion would require changing the JaxAHT ego trainer itself and the mirror assertions in BRDiv, CoMeDi, and LBRDiv, not anything inside DSSE.

2. **AHT only in this framework.** N-agent methods need framework support that the current JaxAHT release does not have. DSSE itself is compatible with a future NAHT extension (nothing about the task is AHT-specific); for now I maintain a NAHT version of DSSE in PyTorch as a separate project.

3. **Return scale is small.** The pipeline normalizes return so absolute numbers are on the order of 10^{-3} to 10^{-2}. The cross-experiment ratios in Table 7 are what to read, not the absolute values.

4. **One teammate-modeling method.** The headline contrast is LIAM versus PPO ego. A small pilot of MELIBA on the encoder-ablation pool at the same 5-seed budget converges at 1/5 seeds (with the converged seed reaching 0.310, in the middle of LIAM's three non-zero seeds at 0.431, 0.474, 0.643); the bimodal pattern that shows up under LIAM appears under MELIBA too, just at a lower convergence frequency. This is consistent with the central claim that the encoder is doing the work, but it suggests convergence reliability depends on the encoder design.

Table 6: BRDiv, CoMeDi, and LBRDiv held-out evaluation returns at $5 \cdot 10^6$ partner steps, $2.5 \cdot 10^6$ ego steps, 1 seed. Held-out IPPO column is the mean over the three seed-123 IPPO partners; heuristics column is the mean over `random_agent`, `greedy_search_agent`, and `sweep_agent`. Numbers are below the noise floor at this budget; the table demonstrates that the integration runs and the checkpoints load, not that any of these methods beats FCP.

Method	Held-out IPPO mean	Heuristics mean	Overall
BRDiv	0.0095	0.0000	0.0048
CoMeDi	0.0060	0.0000	0.0030
LBRDiv	0.0000	0.0000	0.0000

Table 7: LIAM-over-PPO ratio across the four experiments. The direction is preserved in every row; the magnitude varies with how hard the coordination problem is and how many partners the encoder has to distinguish.

Experiment	LIAM / PPO ratio
Encoder ablation, 3 partners	$\sim 9\times$
No-coordination control (NDR$= 1$)	$1.21\times$
3 partners, trained-partner evaluation	$12.9\times$
3 partners, held-out partner evaluation	$6.8\times$
300-partner Fictitious Co-Play	$4.3\times$

A more thorough MELIBA sweep at a larger seed budget is the natural next step.

5. **Matched-backbone PPO vs LIAM.** PPO ego and LIAM ego as shipped in JaxAHT do not share the same actor backbone: PPO ego uses S5 and LIAM ego uses MLP. The no-coordination control rules out a pure architectural explanation for the LIAM-over-PPO gap, but a matched-backbone version of the experiment (either both S5 or both MLP) would give a cleaner same-backbone-plus-encoder vs same-backbone contrast, directly comparable to the encoder-ablation result.

9 Conclusion

This writeup adds DSSE to JaxAHT as a multi-drone search-and-rescue benchmark for AHT, implemented in pure JAX, with a one-line rescue-rule modification (the NDR hyperparameter) that makes the task actually require multi-drone agreement, and a four-experiment validation comparing the teammate-modeling LIAM ego against a vanilla PPO ego baseline, plus an integration check confirming that BRDiv, CoMeDi, and LBRDiv run end to end on the same task. LIAM ego beats PPO ego in every experiment at NDR$= 2$, the gap collapses to $1.21\times$ when the rule is reset to NDR$= 1$, and the encoder-ablation negative control isolates encoder learning (rather than capacity or pipeline noise) as the source of the gap. The pure-JAX backend reaches about 27,000 environment steps per second on one RTX 6000 Ada, which is what makes the full sweep cheap to rerun. Nothing about the benchmark is specific to LIAM as a teammate-modeling method; extending the validation to a full MELIBA run at a larger seed budget is the obvious next step.

Reproducibility statement

Full JaxAHT reproduction from a fresh clone of this repo:

```
pip install -r requirements.txt
export PYTHONPATH=.
bash scripts/regenerate_partner_fixtures.sh
bash scripts/run_phase_e_fcp.sh
python scripts/aggregate_phase_results.py
python writeup/extract_ego_metrics.py
```

The `regenerate_partner_fixtures.sh` step creates the seed-42 and seed-123 IPPO partner checkpoints under `evaluation/fixtures/dsse_ippo_seed{42,123}_pop3` that the validity ablation, the no-coordination control, and the held-out eval YAML reference. If those training directories already exist, the script just refreshes the fixture symlinks without retraining.

The other experiments run from their own scripts in Table 1: `run_phase_a_validity.sh` (validity ablation), `run_phase_b_10seed.sh` (no-coordination control), `run_phase_c_population.sh` (3-partner population), `run_phase_d_heldout.sh` (3-partner held-out eval), and `run_phase_f_brdiv_comedi_lbrdiv.sh` (BRDiv / CoMeDi / LBRDiv integration check). `scripts/run_all_gpu.sh` is an auxiliary wrapper for the qualitative figures (task metrics, episode GIF) and the throughput scaling sweep; it is not a full-sweep orchestrator and does not delegate to the scripts above. Raw Sacred JSON logs for every experiment live under `writeup/logs/`; the rescue-rule unit test (the one that asserts a single drone acting alone cannot trigger a rescue under NDR$= 2$) lives at `tests/test_dsse_wrapper.py`.

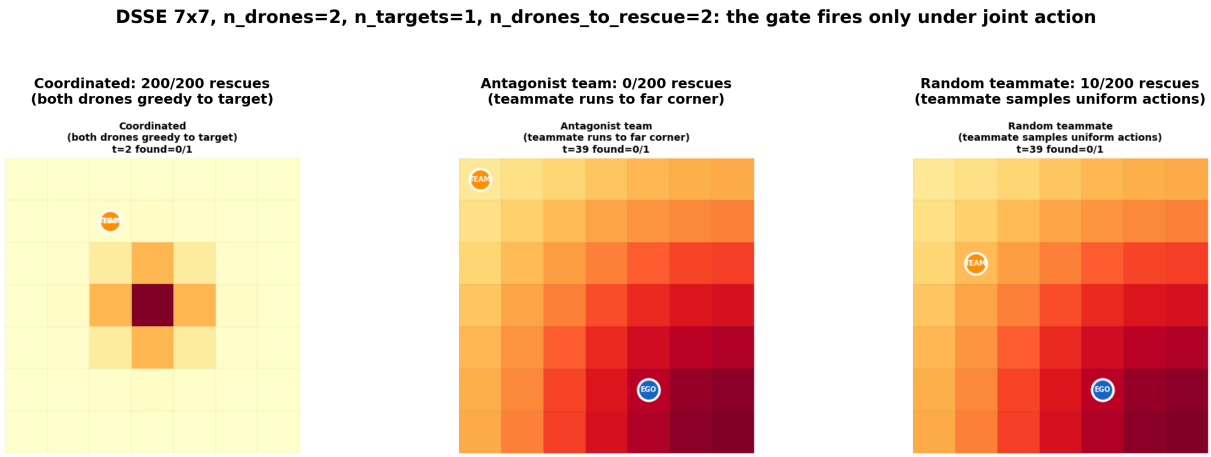
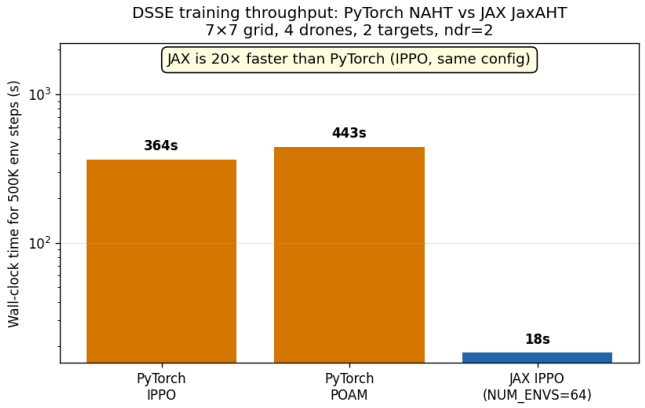
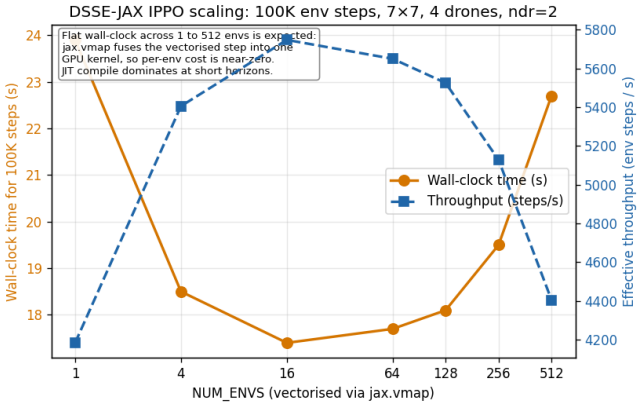


Figure 3: Scripted-policy outcome rates on the 2-drone DSSE variant (g=7, n_t=1, NDR= 2), 200 seeded episodes per scenario. Left: both drones greedy to target, 200/200 rescues. Middle: teammate runs to the opposite corner, 0/200. Right: teammate samples uniform actions, 11/200. The rescue rule only fires under joint action.



(a) Wall-clock speedup: JAX (this repo) vs PyTorch NAHT fork with DSSE.



(b) JAX throughput scaling with NUM_ENVS.

Figure 4: The JAX variant is roughly 20× faster than the PyTorch NAHT-fork DSSE variant at batch size one (see Section 7 for the architectural caveat). Throughput peaks near NUM_ENVS= 64.

References

[1] Stone, P., Kaminka, G. A., Kraus, S., and Rosenschein, J. S. Ad Hoc Autonomous Agent Teams: Collaboration without Pre-Coordination. In *AAAI Conference on Artificial Intelligence*, 2010.

[2] Papoudakis, G., Christianos, F., and Albrecht, S. Agent Modelling under Partial Observability for Deep Reinforcement Learning. In *Neural Information Processing Systems (NeurIPS)*, 2021.

[3] Strouse, D. J., McKee, K. R., Botvinick, M., Hughes, E., and Everett, R. Collaborating with Humans without Human Data. In *Neural Information Processing Systems (NeurIPS)*, 2021.

[4] Wang, C., Hsu, H., and Stone, P. ROTATE: Regret-driven Open-ended Training for Ad Hoc Teamwork. Preprint, 2025.

[5] Laffranchi Falcão, R., Custódio Campos de Oliveira, J., Britto Aragão Andrade, P. H., Ribeiro Rodrigues, R., Jailson Barth, F., and Basso Brancalion, J. F. DSSE: An environment for simulation of RL-empowered drone swarm maritime search and rescue missions. Software, version 0.2.5, 2024. DOI 10.5281/zenodo.12659848.